Assignment 1:

Algorithm Implementation & Empirical Analysis

Workload Distribution:

Member	StudentID	Name	Contribution
1.	s3954789	Shaikh Naima Labiba	Task A + Task B
2.	s3918007	Mihika Pai	Task A + Task B

Task B: Evaluation of Data Structure for different operations

Generation of sub-datasets from a large dataset

We created a Python program to generate 6 datasets from the sampleData200k.txt file. Our dataset sizes are 500, 1250, 6900, 20300, 50000,100000. The following datasets are named as sampData_size.txt with size being the size of the dataset. To generate these datasets, we first created our class called dataset_generator.py. We then imported the module 'random' and created an array to store the sample data from sampleData200k.txt. Using a for loop, these sub-datasets are created by randomly picking the words and their frequencies and storing them in a text file.

Analysis of each Dictionary Type

All the dictionaries below have been tested using the same datasets and test inputs to see which is better and more efficient based on the size of the datasets.

6 test inputs have been made corresponding to the 6 different database sizes. Each test will have 3 outputs indicating which type of dictionary was used. The 6 test inputs are named repTest1.in, repTest2.in, repTest3.in, repTest4.in, repTest5.in and repTest6.in. The test outputs will be named repTest[number]_[dict_type].out. The test inputs were tested with the following:

1. repTest1.in with sampData_500.txt
2. repTest2.in with sampData_1250.txt
3. repTest3.in with sampData_6900.txt
4. repTest4.in with sampData_20300.txt
5. repTest5.in with sampData_50000.txt
6. repTest6.in with sampData_100000.txt

To calculate the runtimes and graph them, we manually checked the execution time using the stopwatch on our smartphones. Even though there are slight differences when manually timing the runtime, we have subtracted 0.25s from the runtime as that is the average speed it takes a human to press a digital button.

Array-based Dictionary:

Runtimes of the array-based dictionary:

Dataset size	**Test input**	**Execution time**
500(small)	repTest1.in	0.13 seconds
1250(small)	repTest2.in	0.18 seconds
6900(medium)	repTest3.in	0.41 seconds

20300(medium)	repTest4.in	0.60 seconds
50000(large)	repTest5.in	0.75 seconds
100000(large)	repTest6.in	0.83 seconds

The time complexity for an Array is O(n)

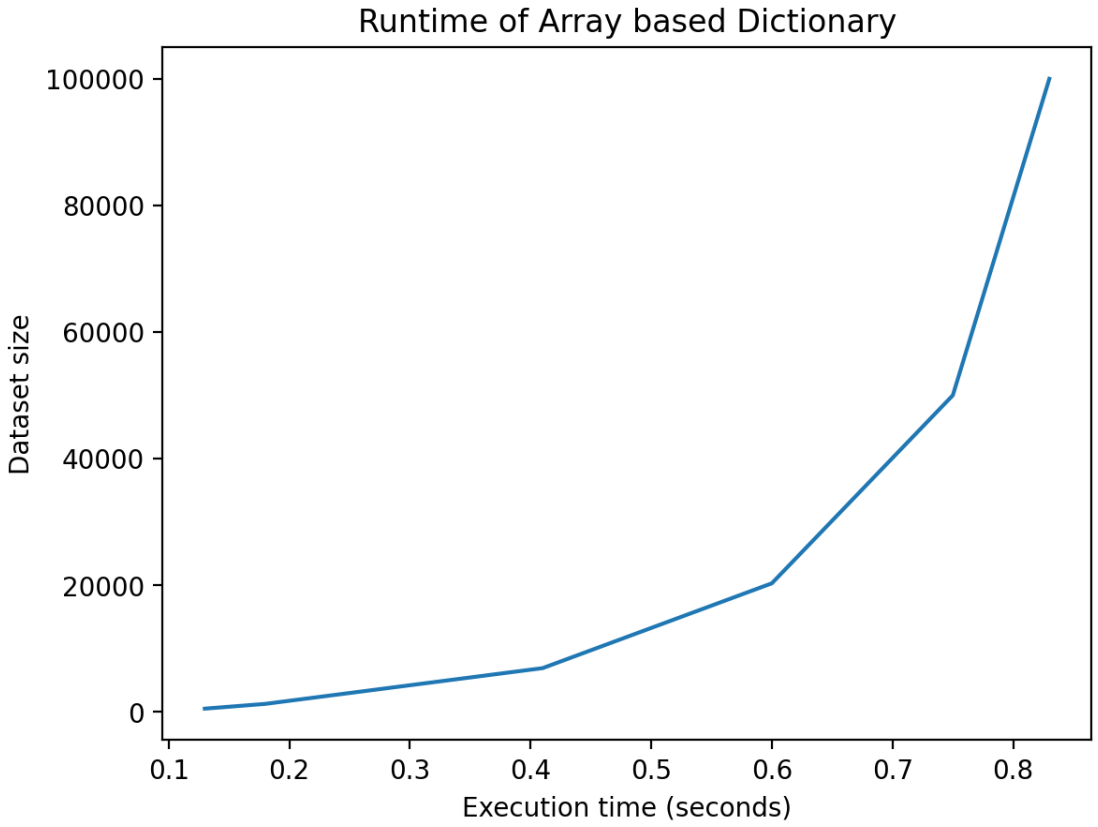


Fig 1

Linked List-based Dictionary:

Runtimes of the Linked List-based Dictionary:

Dataset size	**Test input**	**Execution time**
500(small)	repTest1.in	0.41 seconds
1250(small)	repTest2.in	0.47 seconds
6900(medium)	repTest3.in	2.00 seconds
20300(medium)	repTest4.in	13.6 seconds
50000(large)	repTest5.in	1 minute 26.77 seconds

100000(large)	repTest6.in	6 minutes 2.64 seconds

The time complexity of the Linked List when inserting or deleting an element is always constant, which is O(1), but when searching, the time complexity is O(n). Both these time complexities are the worst, best, and average case time complexities.

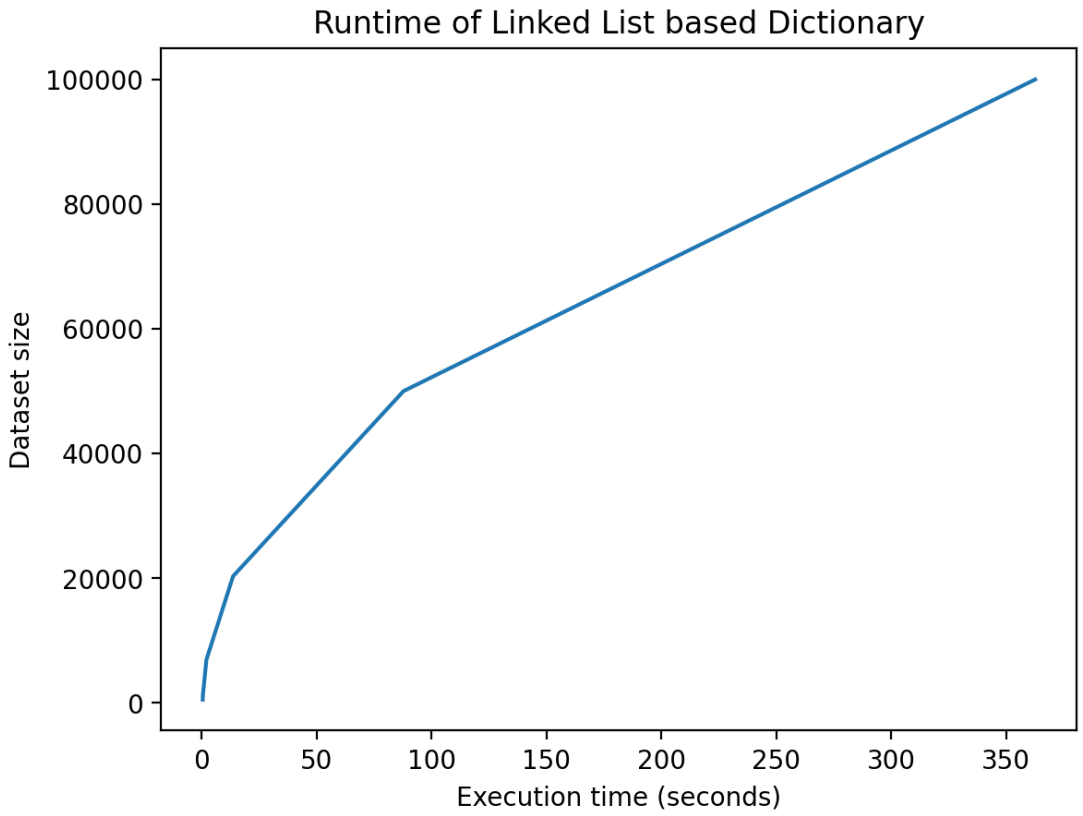


Fig 2

Trie based Dictionary:

Runtimes of the Trie-based Dictionary:

Dataset size	**Test input**	**Execution time**
500(small)	repTest1.in	0.37 seconds
1250(small)	repTest2.in	0.35 seconds
6900(medium)	repTest3.in	0.48 seconds
20300(medium)	repTest4.in	0.66 seconds
50000(large)	repTest5.in	0.71 seconds

100000(large)	repTest6.in	1.78 seconds

The time complexity of Trie is O(n*m), with *n* being the number of words in the tree, and *m* being the length of the longest word.

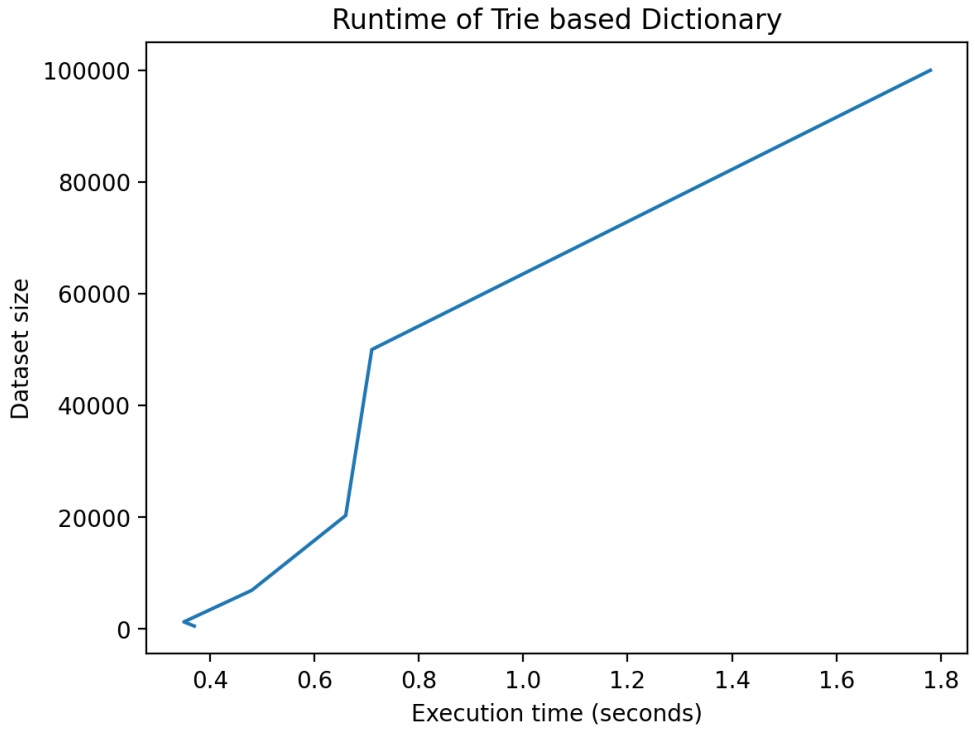


Fig 3

Comparing the execution time of each dictionary type

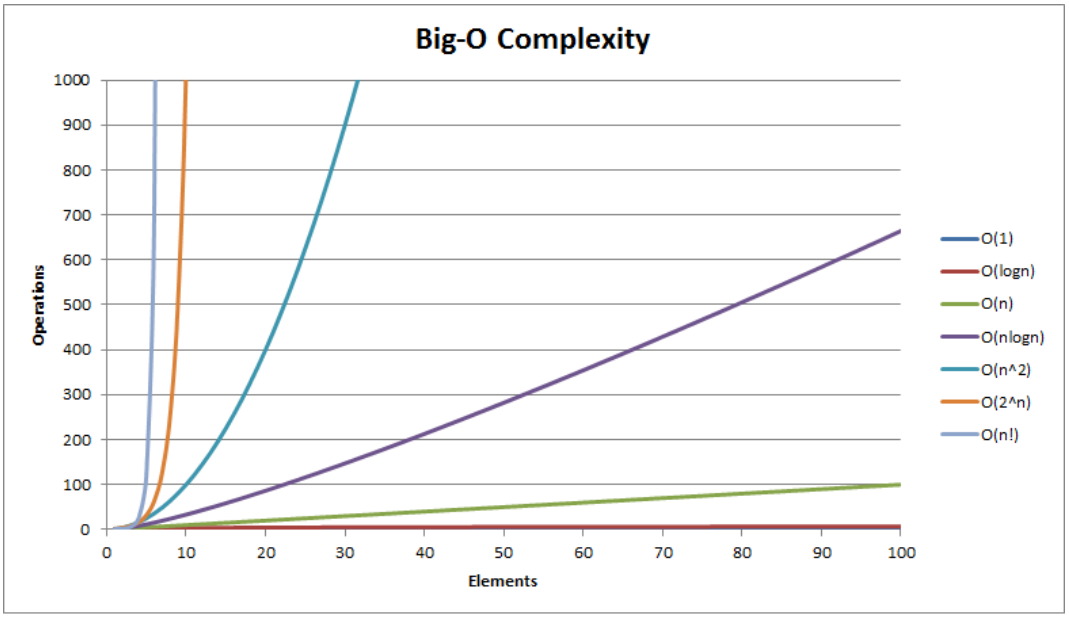


Fig 4

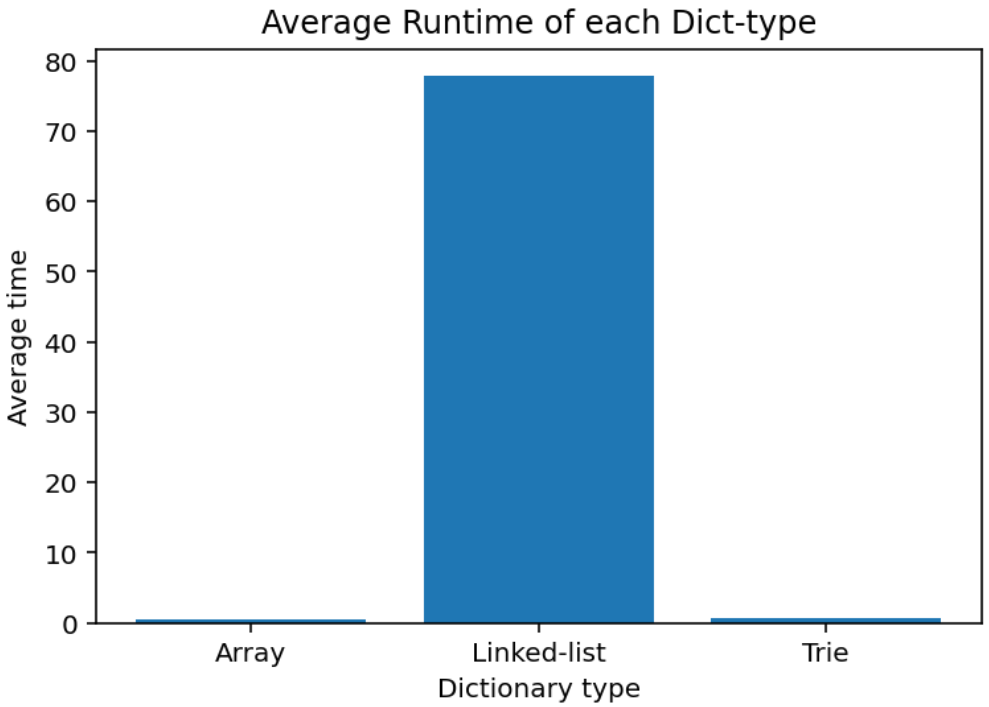


Fig 5

Based on the above graphs, we can conclude the following:

1. The runtime for Linked-list grows exponentially when the database size increases.
2. The time complexity for an Array-based dictionary is $O(2^n)$.
3. The time complexity for a Linked-list based dictionary is $O(n^2)$.
4. The time complexity for a Trie-based dictionary is $O(2^n)$.
5. Trie-based dictionary is the best for large databases, as it processes and outputs the desired output faster than linked-list and array.
6. Linked-list based dictionary is not recommended for large databases as it takes too much time to traverse through all the nodes when searching, adding and deleting elements.

References(if any, will be added below):

Parmar, A. (2021, January 25). *Understanding trie data structure*. Medium. https://medium.com/guidona-softpedia/understanding-trie-data-structure-39e7652114d5

Programiz. (n.d.). *Linked list data structure*. Programiz. https://www.programiz.com/dsa/linked-list#google_vignette

SerHack. (2021, August 14). *How to measure the execution time of a program*. https://serhack.me/articles/measure-execution-time-program/

Natrajan, A. H. (2020, April 4). *How long will my algorithm take?: Understanding time complexity*. Tech 101. https://www.tech101.in/how-long-with-my-algorithm-take-understanding-time-complexity/